

LAPORAN PRAKTIKUM
STRUKTUR DATA
QUEUE

Disusun Oleh :

Nama : Nabila Khairunnisa
Nim : 2511531003
Dosen Pengampu : Dr. Wahyudi, S.T, M.T
Asisten Praktikum : M. Galid Avero



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji Syukur atas kehadiran Allah SWT yang telah melimpahkan Rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum struktur data dengan judul “Queue” dengan baik dan tepat waktu. Dalam menyelesaikan laporan ini saya banyak mendapat arahan dan bimbingan, oleh karena itu saya ingin mengucapkan terimakasih kepada

1. Bapak Dr. Wahyudi, S.T, M.T selaku dosen pengampu
2. Uda M. Galid Averro selaku asisten labor
3. Orang tua yang senantiasa mendoakan
4. Teman teman yang selalu memotivasi

Penulis menyadari bahwa laporan ini jauh dari kata sempurna. Oleh sebab itu, penulis sangat membuka diri apabila ada yang ingin memberikan kritikan dan saran yang sifatnya membantu, penulis akan sangat senang menerima. Tujuannya agar untuk kedepannya bisa menyempurnakan laporan.

Padang, 27 April 2026

Nabila Khairunnisa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	2
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN.....	3
2.1 Deskripsi Praktikum.....	3
2.2 Langkah Langkah Praktikum.....	3
BAB III KESIMPULAN.....	8
3.1 Kesimpulan	8
3.2 Saran	8
DAFTAR PUSTAKA	9
LAPORAN TUGAS PEKAN 4	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Queue atau dalam bahasa Indonesia yang berarti [antrean](#) adalah struktur data yang menyusun elemen-elemen data dalam urutan linier. Prinsip dasar dari struktur data ini adalah “First In, First Out” (FIFO) yang berarti elemen data yang pertama dimasukkan ke dalam antrean akan menjadi yang pertama pula untuk dikeluarkan. [1]

Fungsi struktur data queue yaitu menyimpan data antrean, menentukan urutan Langkah atau proses yang akan dikerjakan oleh system operasi dan system pengelolaan memori dalam menentukan susunan atau urutan memori yang dipakai dan dikeluarkan.

Queue dapat di implementasikan menggunakan array dan LinkedList. Implementasi menggunakan array berperan sebagai wadah penyimpanan berbagai elemen pada antrean. Pada implementasi ini perlu menggunakan dua petunjuk yaitu front dan rear yang menunjukkan posisi elemen pertama dan terakhir. Array mempunyai keuntungan dalam hal akses elemen cepat sesuai indeksnya, tetapi keterbatasannya pada ukuran yang tidak fleksibel.

Implementasi menggunakan LinkedList untuk menyimpan semua elemen pada antrean. Masing-masing simpul mempunyai dua bagian yakni data dan pointer. ada dua penunjuk yang digunakan yaitu front dan rear. Saat ditambahkan elemen baru, Maka simpul baru ditautkan pada simpul terakhir, lalu penunjuk rear diperbarui. Saat elemen dihapus, maka simpul pertama dilepaskan kemudian penunjuk front diperbarui. Linked List memungkinkan adanya penambahan dan penghapusan elemen secara fleksibel, tetapi memori cukup overhead sehingga tidak dapat optimal dalam penyimpanan pointer tambahan. [2]

Contoh nyata dalam kehidupan sehari-hari yang dapat menggambarkan struktur data queue adalah barisan orang yang menunggu untuk membeli tiket di gedung bioskop.

Orang yang baru datang akan bergabung dengan barisan dari ujung dan orang yang berdiri di depan akan menjadi yang pertama mendapatkan tiket dan meninggalkan barisan. Demikian pula dalam struktur data queue, data yang ditambahkan terlebih dahulu akan meninggalkan antrian terlebih dahulu. [3]

1.2 Tujuan Praktikum

1. Mampu memahami konsep dasar struktur data Queue dengan prinsip FIFO (First In First Out)
2. Mampu mengimplementasikan struktur data Queue
3. Mampu membuat dan menggunakan operasi dasar queue
4. Mampu memahami penerapan queue dalam kasus nyata

1.3 Manfaat Praktikum

1. Mahasiswa mampu memahami cara kerja Queue
2. Mahasiswa mampu mengimplementasikan Queue dalam pemrograman java
3. Mahasiswa mampu memahami penerapan Queue dalam kasus nyata
4. Mahasiswa mampu membuat dan menggunakan operasi dasar Queue

BAB II

PEMBAHASAN

2.1 Deskripsi Praktikum

Praktikum Struktur data pada pekan ini adalah membuat Stack yang diimplementasikan menggunakan array melalui class StackArray yang menyediakan operasi dasar seperti push, pop, dan peek. Selanjutnya, class **StackArrayDriver** digunakan sebagai program utama untuk menguji fungsi-fungsi stack. Selain itu, juga terdapat 3 class yang membahas penerapan stack dalam berbagai kasus.

2.2 Langkah Langkah Praktikum

- 1) Buatlah package terlebih dahulu dengan mengklik kanan di folder PrakSD2026_C_2511531003/src , pilih new dan klik package. Setelah itu beri nama pada package tanpa huruf kapital, karakter khusus serta tanpa “space”. lalu “finish”.
- 2) Klik kanan pada package pekan3_2511531003 yang sudah dibuat sebelumnya, pilih “New”, lalu pilih class. Buat nama “QueueArray_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan untuk sebagai alat (data structure) untuk mengatur alur data yang harus diproses secara berurutan .
- 3) Tuliskan program seperti berikut.

```
1 package pekan3_2511531003;
2
3 public class QueueArray_2511531003 {
4     int front, rear, size;
5     int capacity;
6     int array [];
7
8     public QueueArray_2511531003 (int capacity) {
9         this.capacity = capacity;
10        front = this.size = 0;
11        rear = capacity - 1;
12        array = new int [this.capacity];
13    }
14
15    boolean isFull_1003 (QueueArray_2511531003 queue) {
16        return (queue.size == queue.capacity);
17    }
18
19    boolean isEmpty_1003 (QueueArray_2511531003 queue) {
20        return (queue.size == 0);
21    }
22
23    void enqueue_1003 (int item) {
24        if (isFull_1003 (this))
25            return;
26        this.rear = (this.rear + 1) % this.capacity;
27        this.array [this.rear] = item;
28        this.size = this.size + 1;
29        System.out.println (item + " enqueued to queue");
30    }
31 }
```

```

32● int dequeue_1003 () {
33     if (isEmpty_1003 (this))
34         return Integer.MIN_VALUE;
35     int item = this.array [this.front];
36     this.front = (this.front + 1) % this.capacity;
37     this.size = this.size - 1;
38     return item;
39 }
40● int front_1003() {
41     if (isEmpty_1003 (this))
42         return Integer.MIN_VALUE;
43     return this.array [this.front];
44 }
45● int rear_1003 () {
46     if (isEmpty_1003 (this))
47         return Integer.MIN_VALUE;
48     return this.array [this.rear];
49 }
50 //mencetak elemen antrian
51● void display_1003 () {
52     int i;
53     if (front == rear) {
54         System.out.printf("\nAntrian Kosong\n");
55         return;
56     }
57     //kunjungi dari belakang dan cetak
58     for (i = front; i < rear; i++) {
59         System.out.printf(" %d <- ", array [i]);
60     }
61     return;
62 }
63 }
64 }

```

Gambar 2.2.1 kode program

- *Front* digunakan untuk menunjukkan posisi elemen paling depan dalam antrian.
- *Rear* menunjukkan posisi elemen paling belakang.
- *Size* menunjukkan jumlah elemen yang sedang ada dalam antrian.
- *Capacity* menunjukkan kapasitas maksimal antrian.
- *Array* digunakan sebagai tempat menyimpan data antrian.
- Constructor *QueueArray_2511531003 (int capacity)* berfungsi untuk menentukan kapasitas antrian saat objek dibuat. Pada awalnya, *front* dan *size* diisi 0, sedangkan *rear* dan *capacity* -1 agar posisi *rear* bisa bergerak melingkar saat data pertama dimasukkan.
- Method *isFull_1003()* digunakan untuk mengecek apakah antrian sudah penuh. Jika *size* sama dengan *capacity*, maka antrian tidak bisa ditambahkan lagi.
- Method *isEmpty_1003()* digunakan untuk mengecek apakah antrian masih kosong.
- Method *enqueue_1003(int item)* berfungsi untuk menambahkan data ke antrian. Data akan dimasukkan ke posisi belakang atau *rear*. Posisi *rear* dihitung dengan rumus $(rear + 1) \% capacity$, sehingga jika sudah sampai akhir array, posisi bisa kembali ke awal.
- Method *dequeue_1003()* berfungsi untuk menghapus dan mengambil data dari depan antrian. Setelah data diambil, posisi *front* digeser ke elemen berikutnya. Jika antrian kosong, program akan mengembalikan nilai *Integer.MIN_VALUE*.
- Method *front_1003()* digunakan untuk melihat data paling depan tanpa menghapusnya, sedangkan *rear_1003()* digunakan untuk melihat data paling belakang.
- Method *display_1003()* bertujuan untuk menampilkan isi antrian.

- 4) Buat lah class kedua dengan nama “QueueArrayDriver_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu centang tanda “public static void main (string[] args)” lalu “finish”. Class ini digunakan untuk menjalankan operasi pada class *QueueArray_2511531003*.

- 5) Tuliskan program seperti berikut.

```
1 package pekand_2511531003;
2
3 public class QueueArrayDriver_2511531003 {
4     public static void main(String[] args) {
5         QueueArray_2511531003 queue_1003 = new QueueArray_2511531003 (1000);
6         queue_1003.enqueue_1003 (10);
7         queue_1003.enqueue_1003 (20);
8         queue_1003.enqueue_1003 (30);
9         queue_1003.enqueue_1003 (40);
10        System.out.println ("Item di depan " + queue_1003.front_1003());
11        System.out.println ("Item paling belakang " + queue_1003.rear_1003());
12        System.out.println ("tampilkan queue");
13        queue_1003.display_1003();
14        System.out.println ();
15        System.out.println (queue_1003.dequeue_1003() + " dihapus dari queue");
16        System.out.println ("Item di depan: " + queue_1003.front_1003());
17        System.out.println ("Item paling belakang: " + queue_1003.rear_1003());
18        System.out.println ("tampilkan queue setelah satu data dihapus");
19        queue_1003.display_1003();
20    }
21 }
22 }
```

Gambar 2.2.2 Kode Program

- *QueueArray_2511531003 queue_1003 = new QueueArray_2511531003 (1000);* artinya program membuat antrean dengan kapasitas maksimal 1000 data.
 - Method *queue_1003.enqueue* untuk memasukkan beberapa data ke dalam antrean yaitu 10,20,30,40.
 - *System.out.println* untuk menampilkan program
 - *queue_1003.display_1003* digunakan untuk menampilkan isi antrean yang ada.
 - *queue_1003.dequeue_1003()* artinya program menghapus data paling depan dari antrean.
- 6) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
tampilkan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
Item di depan: 20
Item paling belakang: 40
tampilkan queue setelah satu data dihapus
 20 <--  30 <--
```

Gambar 2.2.3 Output program

- 7) Class ketiga dengan nama “ReverseData_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi untuk membalik urutan data dalam queue dengan bantuan stack.
- 8) Tuliskan program seperti berikut.

```

1 package pekan4_2511531003;
2 import java.util.LinkedList;
3
4 public class ReverseData_2511531003 {
5
6     public static void main(String[] args) {
7
8         Queue<Integer> q_1003 = new LinkedList<Integer>();
9         q_1003.add(1);
10        q_1003.add(2);
11        q_1003.add(3); // [1, 2, 3]
12        System.out.println("Sebelum reverse = " + q_1003);
13        Stack<Integer> s = new Stack<Integer>();
14        while (!q_1003.isEmpty()) { // Q -> S
15            s.push(q_1003.remove());
16        }
17        while (!s.isEmpty()) { // S -> Q
18            q_1003.add(s.pop());
19        }
20        System.out.println("Sesudah reverse = " + q_1003); // [3, 2, 1]
21    }
22 }
23
24

```

Gambar 2.2.4 Kode Program

- Method *Queue<Integer> q_1003 = new LinkedList<Integer>();* artinya program membuat sebuah queue menggunakan LinkedList
 - *q_1003.add* untuk menambahkan data
 - *System.out.println* untuk menampilkan program
 - *Stack<Integer> s = new Stack<Integer>();* artinya membuat program menggunakan stack untuk membalik urutan.
 - Method *while (!q_1003.isEmpty())* dan *s.push (q_1003.remove());* artinya semua elemen dipindahkan ke stack.
 - Method *while (!s.isEmpty())* dan *q_1003.add (s.pop());* artinya memindahkan kembali dari stack ke queue lagi.
- 9) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```

<terminated> ReverseData_2511531003 [Java A
Sebelum reverse = [1, 2, 3]
Sesudah reverse = [3, 2, 1]

```

Gambar 2.2.5 Output Program

- 10) Class ke empat dengan nama “QueueLinkedList_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi sebagai class objek/ data untuk penggunaan Queue dengan memanfaatkan LinkedList

11) Tuliskan program seperti berikut

```
1 package pekan4_2511531003;
2
3 import java.util.LinkedList;
4
5 public class QueueLinkedList {
6
7     public static void main(String[] args) {
8         Queue<Integer> q_1003 = new LinkedList<>();
9         //tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
10        for (int i = 0; i < 6; i++)
11            q_1003.add(i);
12        //Menampilkan isi antrian
13        System.out.println("Elemen Antrian " + q_1003);
14        //Untuk menghapus kepala antrian
15        int hapus = q_1003.remove();
16        System.out.println("Hapus Elemen = " + hapus);
17        System.out.println(q_1003);
18        //Untuk melihat antrian terdepan
19        int depan = q_1003.peek();
20        System.out.println("Kepala Antrian = " + depan);
21
22        int banyak = q_1003.size();
23        System.out.println("Size Antrian " + banyak);
24    }
25 }
26
27 }
```

Gambar 2.2.6 Kode Program

- Method *q_1003.add* untuk menambahkan data ke queue
- *Iterator<String> iterator = q_1003.iterator()*; digunakan untuk mengakses elemen satu persatu dari queue.
- *has.Next* untuk mengecek apakah masih ada elemen berikutnya.
- *next()* untuk mengambil elemen satu per satu.

12) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> IterasiQueue_2511531003 [Java Application] C:\Users\|
Praktikum Struktur Data Dan Algoritma
```

Gambar 2.2.7 Output Program

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa queue bekerja dengan prinsip FIFO (First In First Out), dimana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Implementasi queue dapat dilakukan menggunakan array dan LinkedList.

Pada queue berbasis array, diperlukan pengaturan indeks seperti front dan rear agar penggunaan memori lebih efisien. Sedangkan pada queue berbasis LinkedList, pengelolaan elemen menjadi lebih fleksibel karena tidak memiliki kapasitas tetap.

3.2 Saran

Untuk menambah pengembangan praktikum di masa mendatang disarankan untuk mengoptimalkan fungsi display pada queue array agar dapat menangani kondisi circular queue dengan lebih akurat. Selain itu disarankan untuk mencoba implementasi queue dengan metode lain untuk memperluas pemahaman.

DAFTAR PUSTAKA

- [1] R. maulana, "Struktur Data Queue: Pengertian, Fungsi, dan Jenisnya," dicoding, 29 November 2023. [Online]. Available: <https://www.dicoding.com/blog/struktur-data-queue-pengertian-fungsi-dan-jenisnya/>. [Accessed 29 April 2026].
- [2] Team, "Struktur Data Queue: Pengertian, Fungsi, dan Jenisnya," codingstudio, 15 Maret 2024. [Online]. Available: <https://codingstudio.id/blog/struktur-data-queue-adalah/>. [Accessed 29 April 2026].
- [3] Trivusi, "Struktur Data Queue: Pengertian, Jenis, dan Kegunaannya," trivusi, 1 Juli 2023. [Online]. Available: <https://www.trivusi.web.id/2022/07/struktur-data-queue.html>. [Accessed 29 April 2026].

LAPORAN TUGAS PRAKTIKUM PEKAN 4

1. KODE PROGRAM

//Class 1 : Queue_2511531003

```
package pekan4_2511531003;
```

```
public class Queue_2511531003 {  
    int front, rear, max;  
    String[] queue;
```

```
    public Queue_2511531003(int max) {  
        this.max = max;  
        queue = new String[max];  
        front = -1;  
        rear = -1;  
    }
```

```
    boolean isEmpty_1003() {  
        return front == -1;  
    }
```

```
    boolean isFull_1003() {  
        return rear == max - 1;  
    }
```

```
    void enqueue_1003(String data) {  
        if (isFull_1003()) {  
            System.out.println("Antrian penuh!");  
        } else {  
            if (isEmpty_1003()) {  
                front = 0;  
            }  
            rear++;  
            queue[rear] = data;  
            System.out.println("Data berhasil ditambahkan ke antrian");  
        }  
    }
```

```

void dequeue_1003() {
    if (isEmpty_1003()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println(queue[front] + " telah dilayani");

        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front++;
        }
    }
}

void display_1003() {
    if (isEmpty_1003()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Isi antrian:");
        for (int i = front; i <= rear; i++) {
            System.out.println((i - front + 1) + ". " + queue[i]);
        }
    }
}

void reverse_1003() {
    if (isEmpty_1003()) {
        System.out.println("Antrian kosong!");
    } else {
        int kiri = front;
        int kanan = rear;

        while (kiri < kanan) {
            String temp = queue[kiri];
            queue[kiri] = queue[kanan];
            queue[kanan] = temp;

            kiri++;

```

```

        kanan--;
    }

    System.out.println("Antrian berhasil dibalik");
}
}
}

```

//Class 2 : AntrianLoket_2511531003

```

package pekan4_2511531003;
import java.util.Scanner;
public class AntrianLoket_2511531003 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Queue_2511531003 q_1003 = new Queue_2511531003(10);

        int pilih;
        String nama;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilih = input.nextInt();
            input.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    nama = input.nextLine();
                    q_1003.enqueue_1003(nama);
                    break;

```

```

        case 2:
            q_1003.dequeue_1003();
            break;

        case 3:
            q_1003.display_1003();
            break;

        case 4:
            q_1003.reverse_1003();
            q_1003.display_1003();
            break;

        case 5:
            System.out.println("Program selesai.");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
    }

    } while (pilih != 5);

    input.close();
}
}

```

2. SCREENSHOT CODE PROGRAM



```

1 package pekan4_2511531003;
2
3 public class Queue_2511531003 {
4     int front, rear, max;
5     String[] queue;
6
7     public Queue_2511531003(int max) {
8         this.max = max;
9         queue = new String[max];
10        front = -1;
11        rear = -1;
12    }
13
14    boolean isEmpty_1003() {
15        return front == -1;
16    }
17
18    boolean isFull_1003() {
19        return rear == max - 1;
20    }
21
22    void enqueue_1003(String data) {
23        if (isFull_1003()) {
24            System.out.println("Antrian penuh!");
25        } else {
26            if (isEmpty_1003()) {
27                front = 0;
28            }
29            rear++;
30            queue[rear] = data;
31            System.out.println("Data berhasil ditambahkan ke antrian");
32        }
33    }
34 }

```

```

    void dequeue_1003() {
        if (isEmpty_1003()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue[front] + " telah dilayani");

            if (front == rear) {
                front = -1;
                rear = -1;
            } else {
                front++;
            }
        }
    }

    void display_1003() {
        if (isEmpty_1003()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            for (int i = front; i <= rear; i++) {
                System.out.println((i - front + 1) + ". " + queue[i]);
            }
        }
    }

    void reverse_1003() {
        if (isEmpty_1003()) {
            System.out.println("Antrian kosong!");
        } else {
            int kiri = front;
            int kanan = rear;

            while (kiri < kanan) {
                String temp = queue[kiri];
                queue[kiri] = queue[kanan];
                queue[kanan] = temp;

                kiri++;
                kanan--;
            }

            System.out.println("Antrian berhasil dibalik");
        }
    }
}

```

3. OUTPUT

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian

```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
Isi antrian:
1. Budi
2. Andi
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
```

4. PENJELASAN TUGAS

Pada tugas pekan 4 ini, diperintahkan untuk membuat sebuah program dengan implementasi Queue (FIFO) menggunakan array dalam bahasa java. Pada tugas ini terdapat studi kasus yang digunakan yaitu antrian pelanggan di loket pelayanan. Program ini terdiri dari 2 kelas yaitu kelas Queue_2511531003 dan kelas AntrianLoket_2511531003.

Pada kelas **Queue_2511531003** merupakan kelas utama yang digunakan untuk mengelola data antrian. Didalam ini terdapat seluruh logika dan operasi queue berbasis array. Di dalam nya terdapat atribut (seperti front, rear, max dan queue[]), constructor yang digunakan untuk menginisialisasi queue dan method (seperti isEmpty(), isFull(), enqueue(), dequeue(), display(), reverse()).

Pada kelas **AntrianLoket_2511531003** merupakan kelas driver yang digunakan untuk menjalankan program. Kelas ini memiliki fungsi utama yaitu menampilkan menu program, menerima input dari pengguna dan memanggil method **Queue_2511531003**. Alur program nya yaitu membuat objek queue kemudian menampilkan menu dan menggunakan switch-case untuk menjalankan pilihan user.